

Seguridad de la Información

Práctica 4: Correo Electrónico Seguro S/MIME

TITULACIÓN: Grado en Ingeniería Informática

AUTOR: Sergio Acosta Quintana

Fecha: 7 de marzo de 2026

Índice general

1. Envío, recepción y decodificación manual de mensajes S/MIME	2
1.1. Creación de nuestro certificado personal	2
1.2. Instalación de los certificados en el cliente de correo e intercambio de mensajes S/MIME	4
1.3. Decodificación con la utilidad “openssl smime” del mensaje cifrado y firmado que recibimos	12

Capítulo 1

Envío, recepción y decodificación manual de mensajes S/MIME firmados y cifrados, empleando certificados creados por el estudiante y firmados con el certificado raíz de prácticas de esta asignatura

En este documento se explica y demuestra paso a paso la cuarta práctica de la asignatura. Para poner un poco de contexto teórico antes de empezar, hay que destacar que S/MIME es un estándar basado en criptografía de clave pública que nos permite garantizar la confidencialidad, la integridad y la autenticidad en los correos electrónicos. Básicamente, asegura que nadie pueda leer el correo por el medio, que no se modifique, y que sepamos al cien por cien quién lo ha enviado.

En las siguientes páginas aprenderemos cómo hacer esto primero como un usuario normal usando el gestor Mozilla Thunderbird, y luego pasaremos a analizar y decodificar estos mensajes a mano en la consola usando la herramienta OpenSSL.

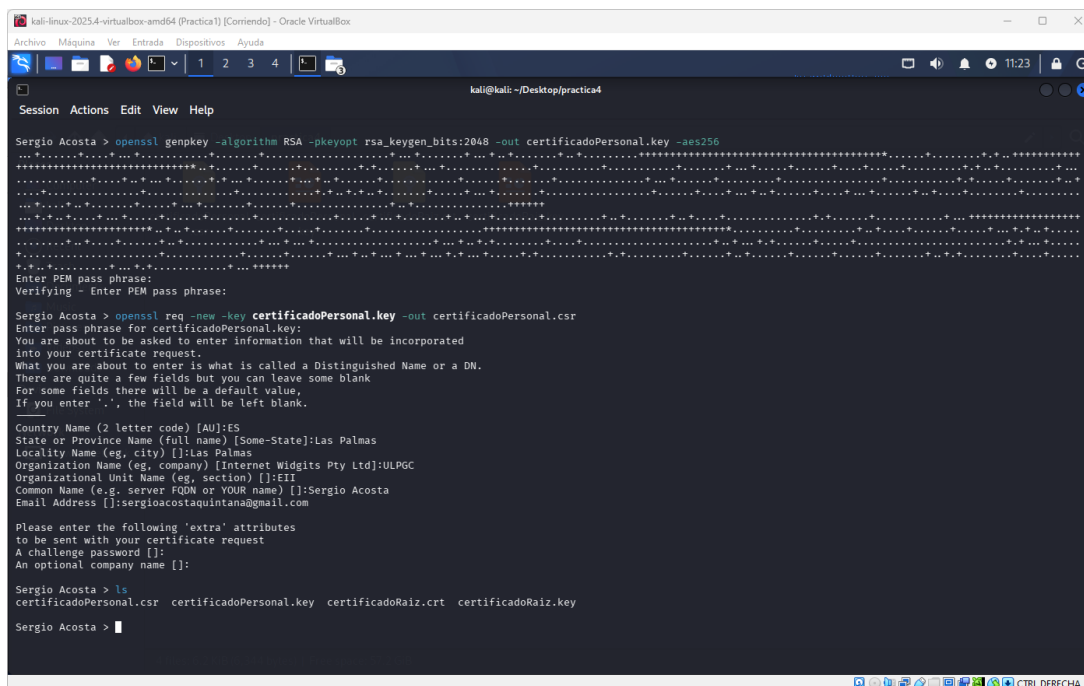
Como paso previo, antes de empezar a meter comandos criptográficos en la máquina virtual, hemos personalizado el entorno de consola como pide el guion de la práctica ejecutando:

```
export PS1="Sergio Acosta > "
```

1.1. Creación de nuestro certificado personal

En esta primera parte vamos a generar nuestro propio material criptográfico, usando como base el certificado raíz que nos dan en la asignatura (`certificadoRaiz.crt` y su clave privada).

Como podemos ver en la figura 1.1, primero ejecutamos el comando para crear nuestra clave privada (`certificadoPersonal.key`) usando el algoritmo RSA y protegiéndola con cifrado AES-256 para añadir una buena capa de seguridad. Después, generamos el fichero de solicitud CSR, teniendo bastante cuidado de rellenar bien los campos, sobre todo el *Common Name (CN)* con nuestro nombre y el *Email Address* con el correo que vamos a usar.



```
kali@kali: ~/Desktop/practica4
Sergio Acosta > openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out certificadoPersonal.key -aes256
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:

Sergio Acosta > openssl req -new -key certificadoPersonal.key -out certificadoPersonal.csr
Enter pass phrase for certificadoPersonal.key:
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:ES
State or Province Name (full name) [Some-State]:Las Palmas
Locality Name (eg, city) []:Las Palmas
Organization Name (eg, company) [Internet Widgits Pty Ltd]:ULPGC
Organizational Unit Name (eg, section) []:EII
Common Name (e.g. server FQDN or YOUR name) []:Sergio Acosta
Email Address []:sergioacostaquintana@gmail.com

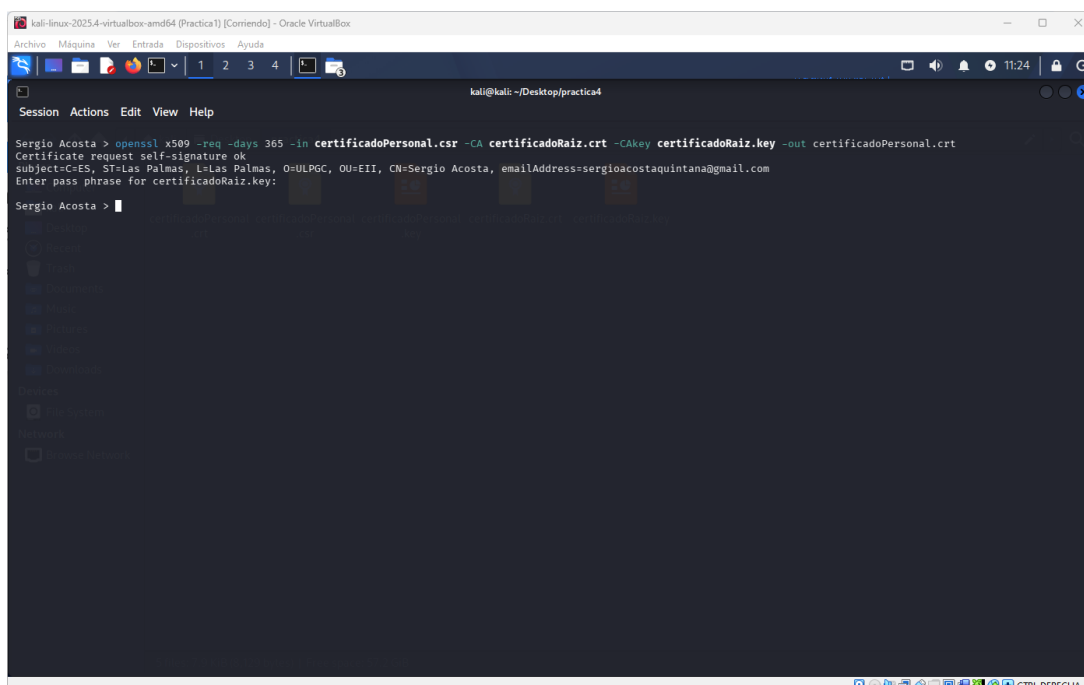
Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:
An optional company name []:

Sergio Acosta > ls
certificadoPersonal.csr  certificadoPersonal.key  certificadoRaiz.crt  certificadoRaiz.key

Sergio Acosta >
```

Figura 1.1: Generación de la clave privada con protección por contraseña y creación del fichero de solicitud CSR.

Una vez tenemos el CSR, pasamos a firmarlo usando la CA de la asignatura. En la figura 1.2 podemos comprobar cómo se emite correctamente nuestro certificado público definitivo (`certificadoPersonal.crt`), dándole una validez de 365 días.



```
kali@kali: ~/Desktop/practica4
Sergio Acosta > openssl x509 -req -days 365 -in certificadoPersonal.csr -CA certificadoRaiz.crt -CAkey certificadoRaiz.key -out certificadoPersonal.crt
Certificate request self-signature ok
subject=C=ES, St=Las Palmas, L=Las Palmas, O=ULPGC, OU=EII, CN=Sergio Acosta, emailAddress=sergioacostaquintana@gmail.com
Enter pass phrase for certificadoRaiz.key:

Sergio Acosta >
```

Figura 1.2: Proceso de firma del CSR mediante la Autoridad de Certificación.

Para terminar esta parte, y para que Thunderbird (o cualquier otro cliente de correo) pueda importar nuestra clave y nuestro certificado a la vez, empaquetamos los dos ficheros

juntos en un contenedor seguro con formato PKCS#12 (`certificadoPersonal.p12`). Le ponemos una contraseña de exportación para protegerlo bien, tal y como se documenta en la figura 1.3.

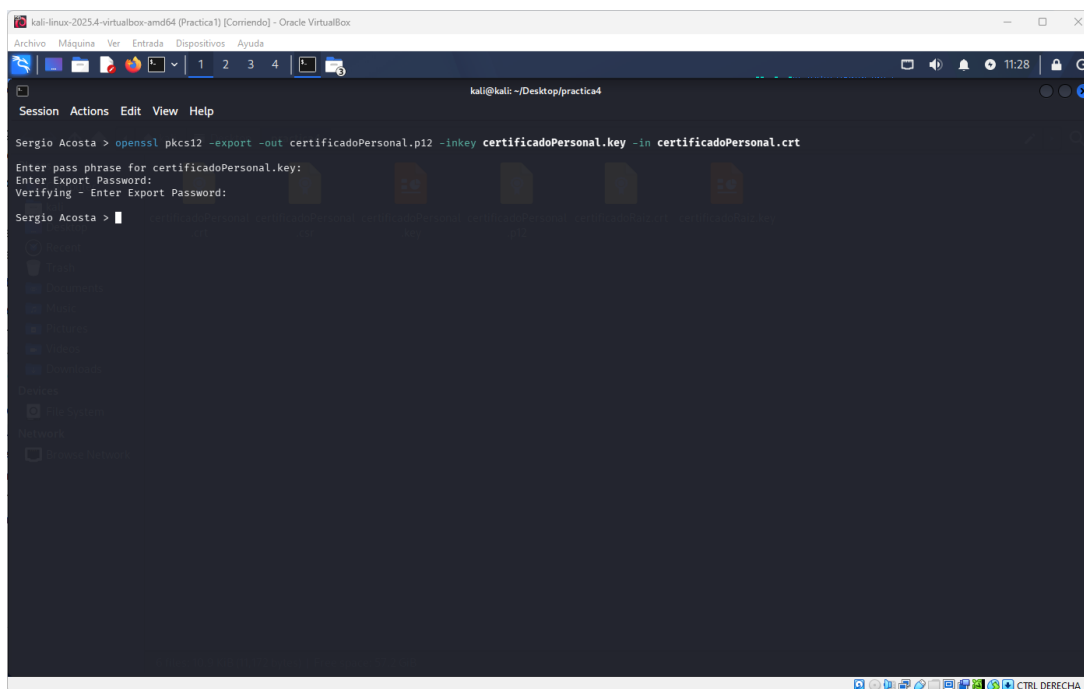


Figura 1.3: Empaquetado de los certificados en formato PKCS#12 para su posterior importación en Thunderbird.

1.2. Instalación de los certificados en el cliente de correo e intercambio de mensajes S/MIME

Ahora nos pasamos a Mozilla Thunderbird para configurarlo todo. Lo primero que hacemos es abrir el programa y configurar nuestra cuenta de correo básica vinculada. En las figuras 1.4 y 1.5 podemos ver la pantalla principal del gestor de correo recién arrancado y listo para empezar a recibir mensajes en nuestra bandeja de entrada.

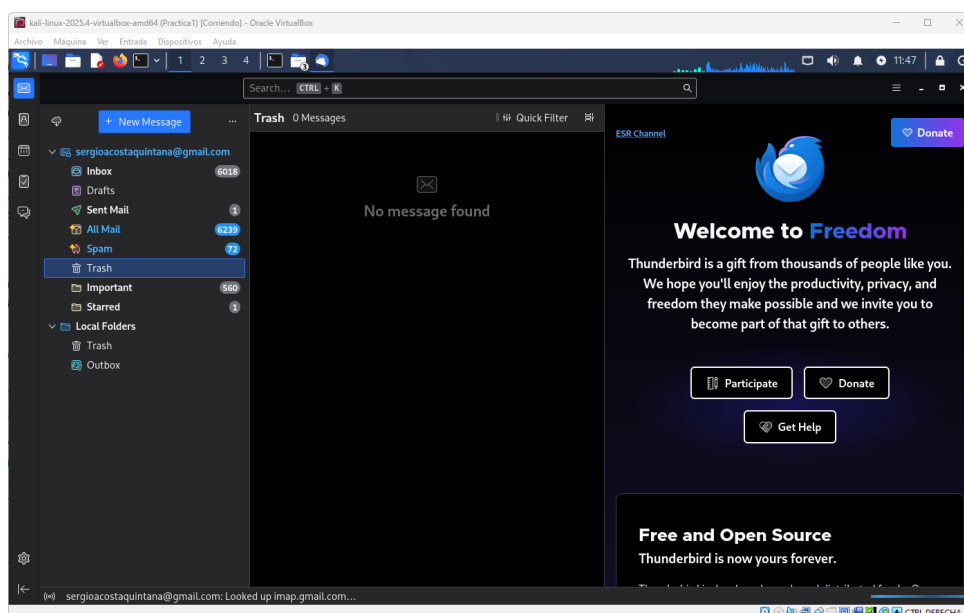


Figura 1.4: Pantalla de inicio de Mozilla Thunderbird tras configurar la cuenta de correo.

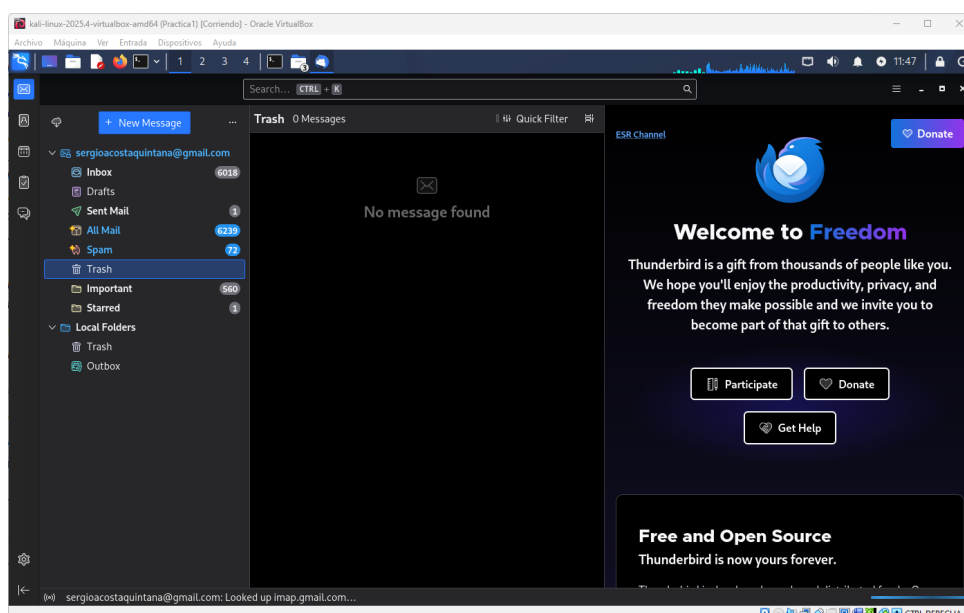


Figura 1.5: Bandeja de entrada principal de Thunderbird lista para operar.

A continuación, nos dirigimos a la configuración interna del programa para añadir nuestros certificados. En la figura 1.6 se aprecia cómo entramos en la pestaña de Ajustes (Settings), navegamos hasta la sección de "Privacy & Security", y bajamos hasta el apartado de "Certificates" para abrir el gestor.

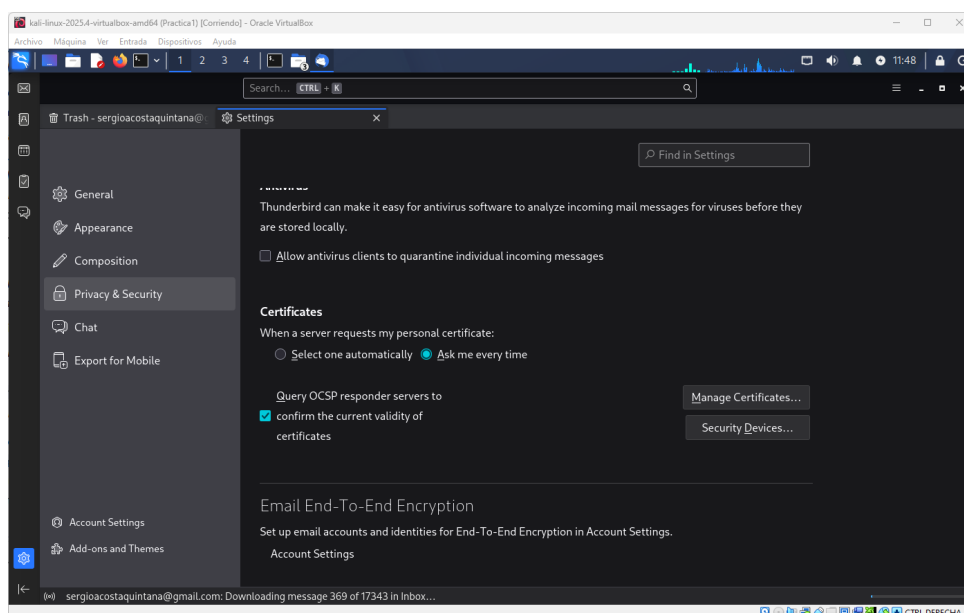


Figura 1.6: Sección de Privacidad y Seguridad dentro de los ajustes de Thunderbird.

Una vez dentro del Certificate Manager, nos vamos a la pestaña de Autoridades (Authorities), como se muestra en la figura 1.7. Aquí es donde tenemos que importar la clave pública de la Autoridad de Certificación de la asignatura (`certificadoRaiz.crt`).

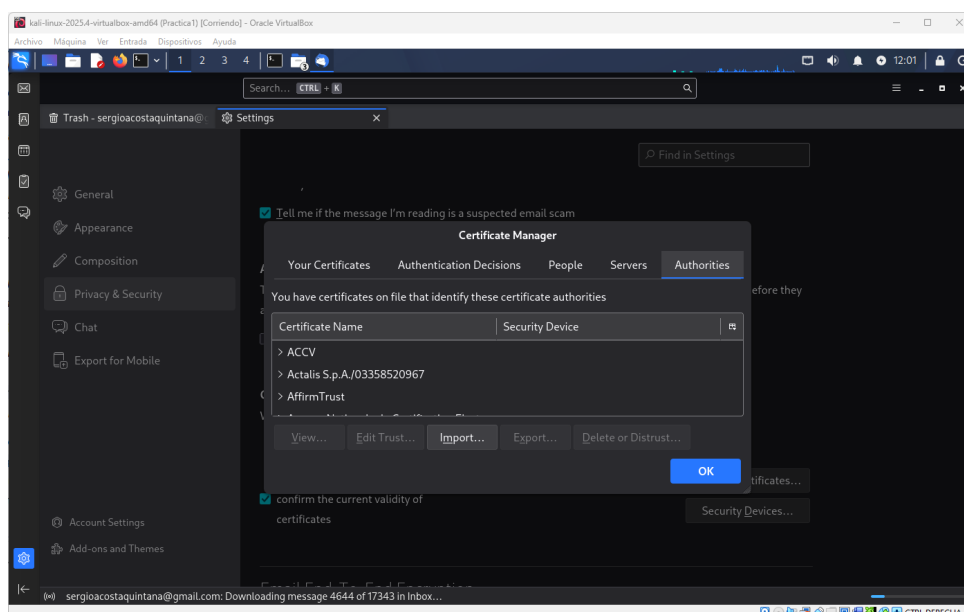


Figura 1.7: Pestaña de Autoridades en el gestor de certificados antes de la importación.

Al importarla, salta una ventana crucial. Como se ve en la figura 1.8, es súper importante marcar la casilla "Trust this CA to identify websites/email users" para darle confianza explícita. Si no lo hacemos, la cadena de confianza fallará más adelante cuando intentemos enviar o recibir correos.

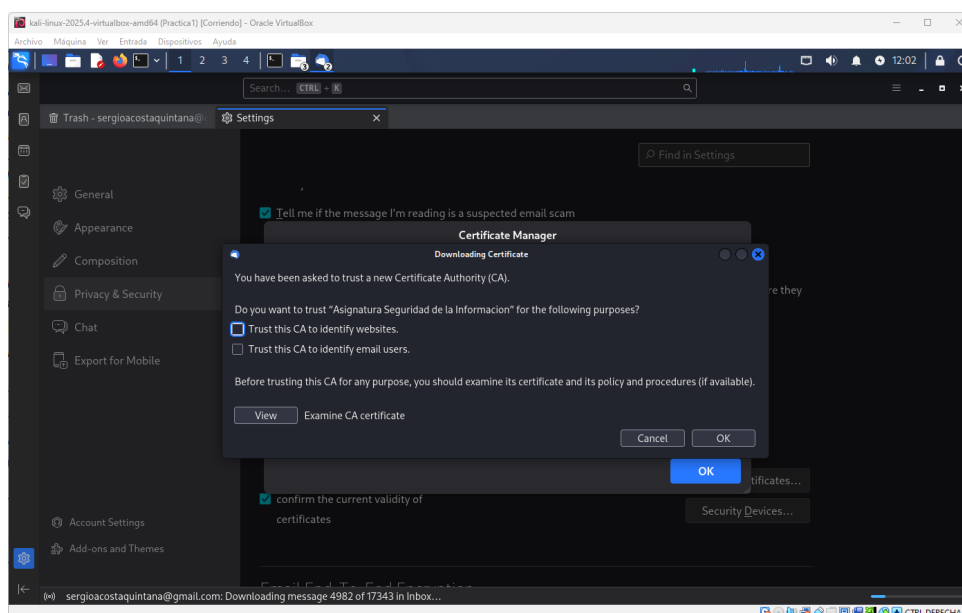


Figura 1.8: Asignación de confianza a la Autoridad de Certificación de la asignatura.

Si comprobamos los detalles de esta Autoridad recién importada, tal y como vemos en la figura 1.9, podemos confirmar que toda la información del emisor concuerda con la proporcionada por la ULPGC para esta práctica.

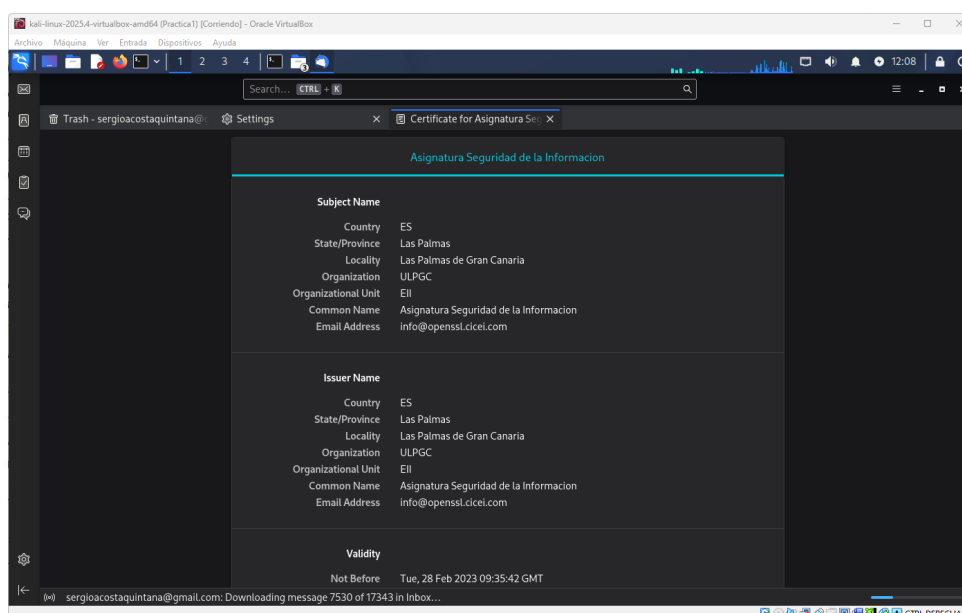


Figura 1.9: Detalles del certificado de la Autoridad Raíz (CA) de la asignatura importado correctamente.

Después importamos nuestro contenedor `certificadoPersonal.p12` en la pestaña de certificados propios. En la figura 1.10 podemos ver que Thunderbird lo reconoce sin problemas y nos muestra todos los detalles que metimos antes al crearlo en la terminal.

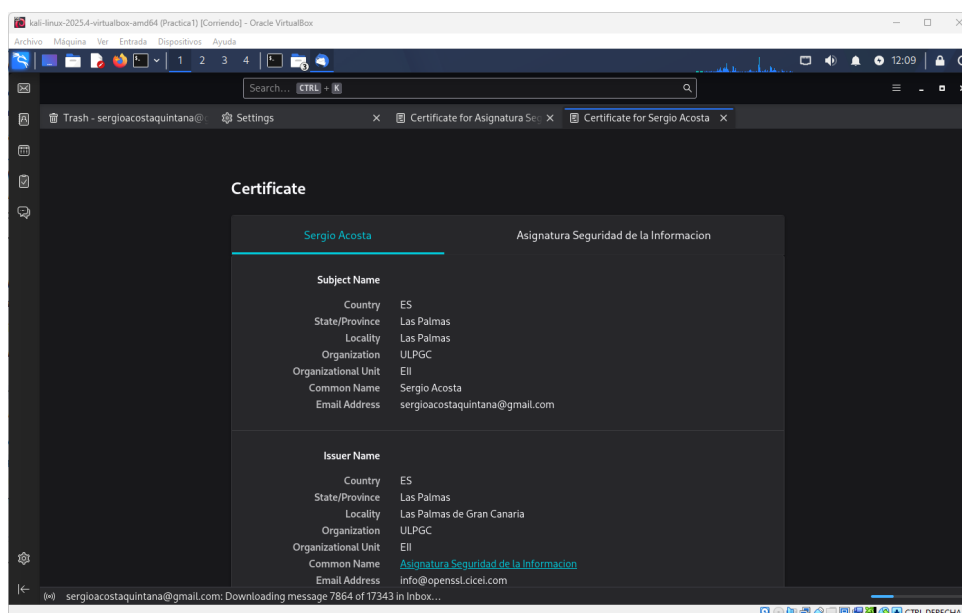


Figura 1.10: Visualización de los detalles de nuestro certificado personal importado en Thunderbird.

Con los certificados ya instalados, nos vamos a los ajustes de la cuenta, a la sección de *End-To-End Encryption*. Como vemos en la figura 1.11, seleccionamos explícitamente nuestro certificado para usarlo tanto para firmar digitalmente como para cifrar nuestros correos S/MIME.

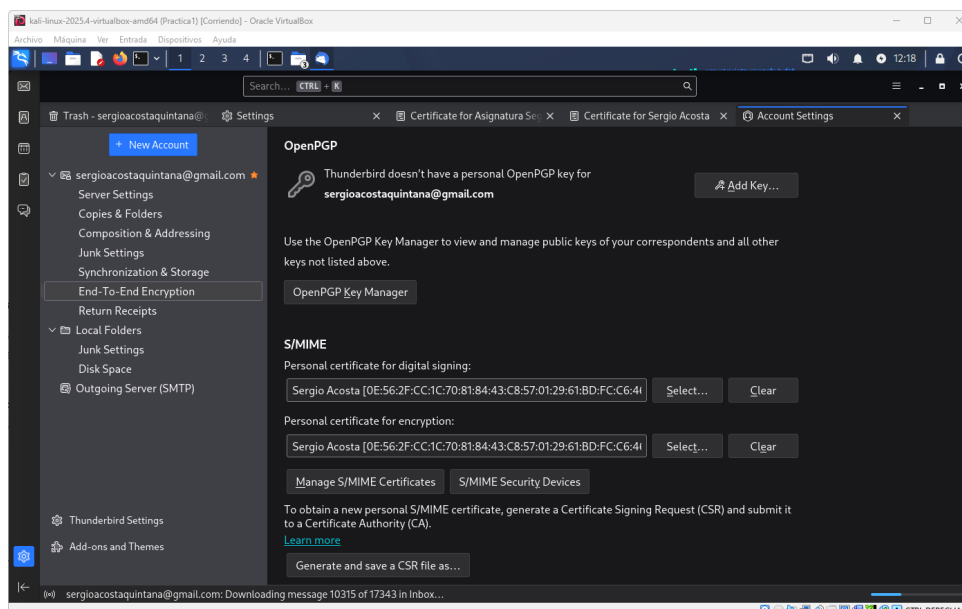


Figura 1.11: Selección del certificado personal en los ajustes de S/MIME de la cuenta.

Como para poder mandarle un correo cifrado a un compañero necesitamos tener su clave pública primero, hacemos un intercambio previo. Le mandamos un primer correo marcando solo la opción de firma digital, como se muestra en la figura 1.12.

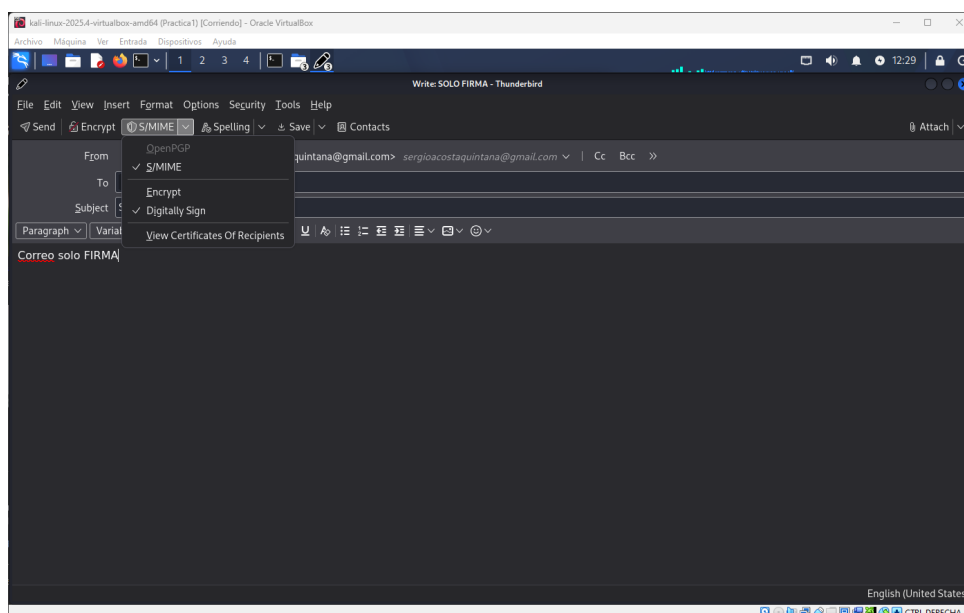


Figura 1.12: Redacción del mensaje inicial activando exclusivamente la opción de firma digital.

Al enviarlo, en la figura 1.13 vemos cómo nos salta una ventana pidiendo identificarnos con nuestro certificado frente al servidor SMTP de Gmail (esto actúa como una validación de cliente TLS), confirmando que usamos el certificado que nos firmó la CA de la asignatura.

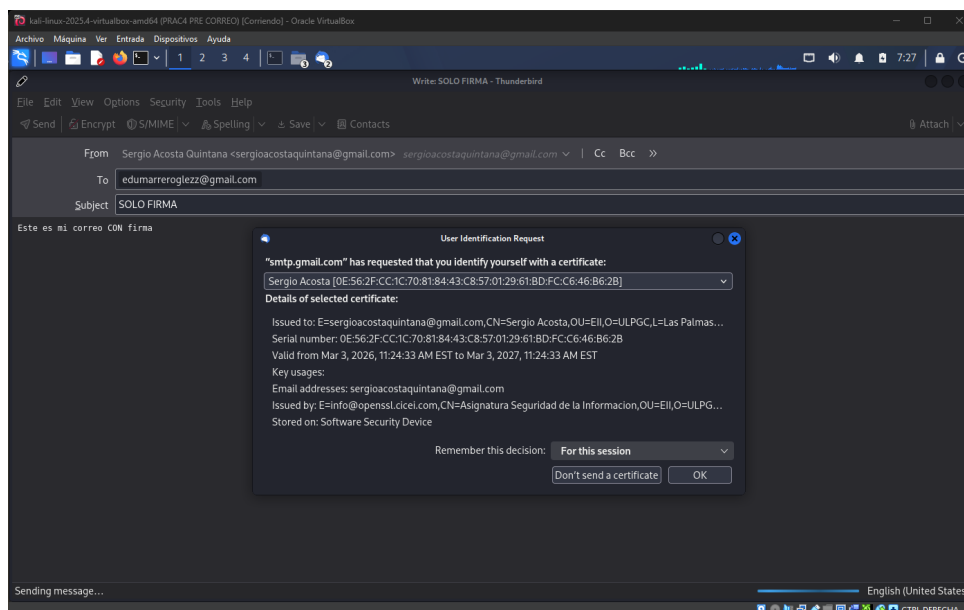


Figura 1.13: Confirmación del certificado a utilizar durante el envío de la comunicación.

Cuando recibimos la respuesta del compañero (Eduardo Marrero), que también viene firmada, Thunderbird es listo y extrae su clave pública del mensaje, guardándola automáticamente en el gestor de certificados. En la figura 1.14 podemos comprobar que el certificado de Eduardo ya está importado correctamente.

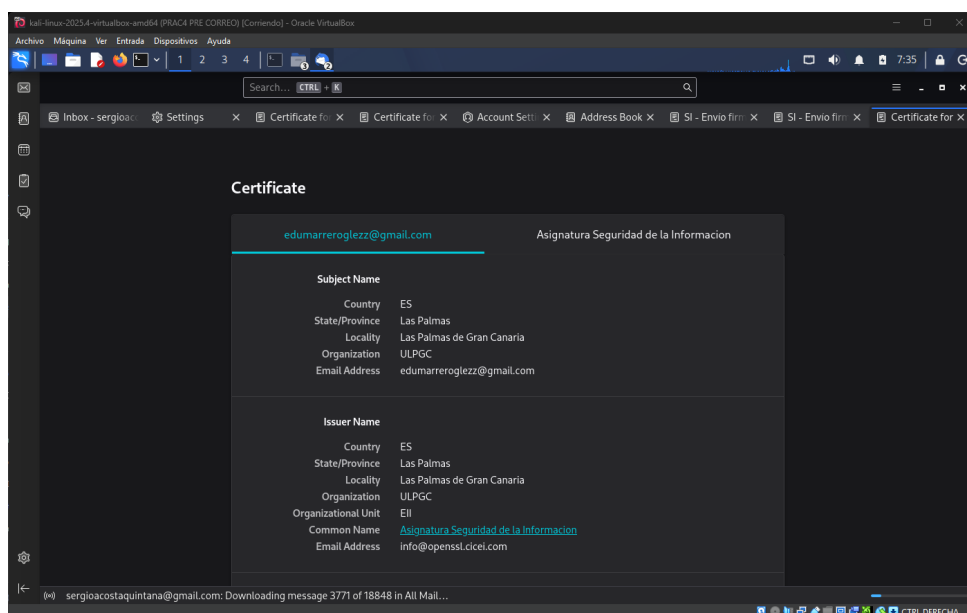


Figura 1.14: Certificado público del compañero importado automáticamente en nuestra libreta.

Si miramos el código fuente de este primer correo recibido (figura 1.15), podemos apreciar cómo el texto viaja en plano pero va acompañado de la firma criptográfica en la parte inferior. Como detalle técnico importante que exige el estándar, esta firma está codificada en **base64**. Esto se hace así porque los algoritmos generan datos en crudo (binarios), y para que puedan viajar sin corromperse por los protocolos clásicos de correo, se convierten a texto mediante base64 y se empaquetan siguiendo la estructura delimitada de **PEM** (empezando por **MIAGC...**), lo que permite diferenciar claramente el mensaje del bloque seguro S/MIME.

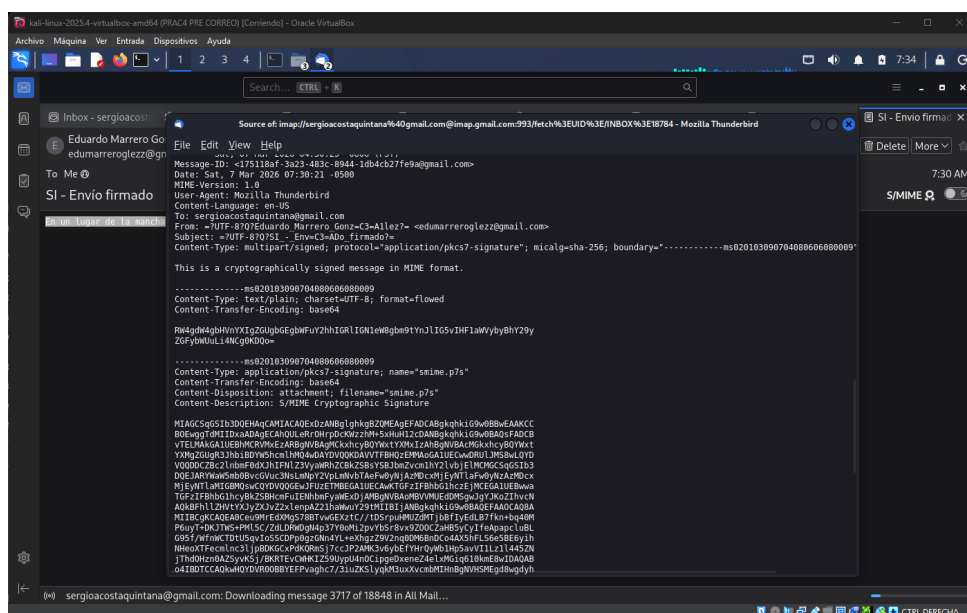


Figura 1.15: Análisis del código fuente del primer mensaje firmado recibiendo evidenciando la firma en base64.

Finalmente, como ya tenemos su clave pública, vamos a mandarle el correo definitivo. Thunderbird ya nos avisa en la parte inferior de que podemos usar cifrado extremo a extremo (figura 1.16). Esta vez activamos las dos opciones a la vez: cifrar y firmar. El uso de S/MIME nos permite garantizar aquí las tres propiedades fundamentales de seguridad: confidencialidad (mediante cifrado), integridad del mensaje y autenticidad del remitente (mediante firma digital).

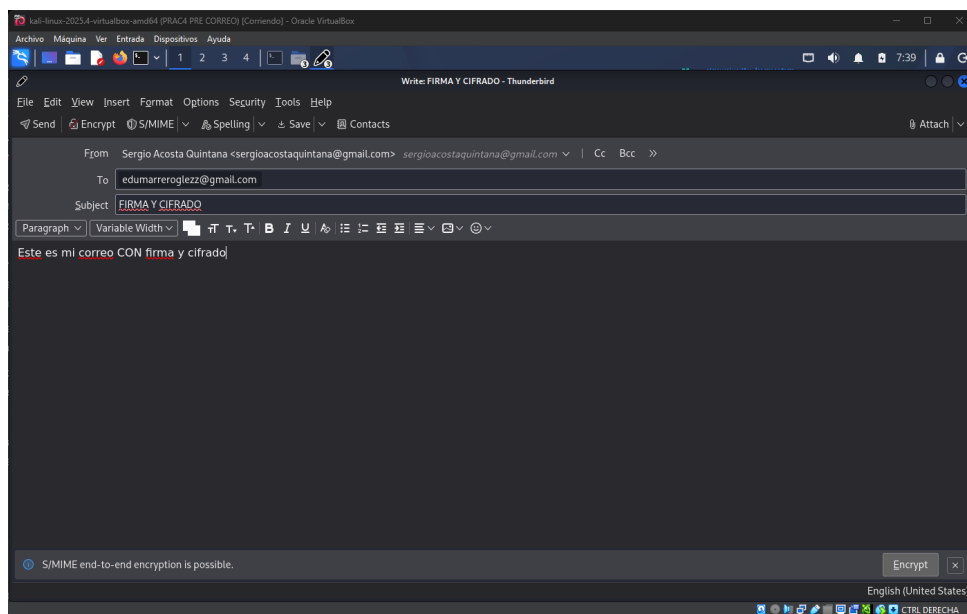


Figura 1.16: Notificación de disponibilidad de cifrado S/MIME al disponer de la clave pública del destinatario.

En la figura 1.17 vemos cómo queda la redacción del correo con las dos opciones marcadas en el menú de seguridad justo antes de darle a enviar.

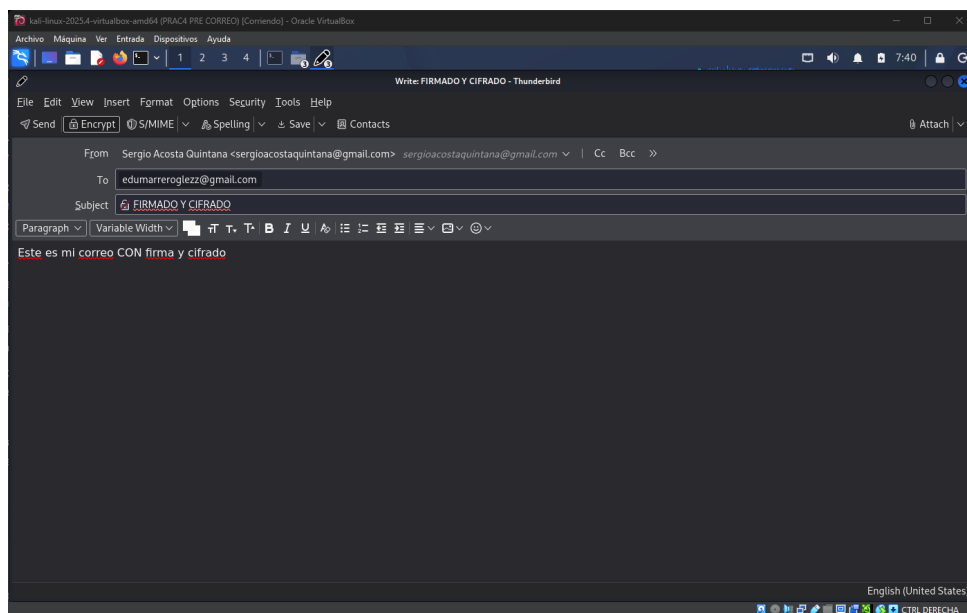


Figura 1.17: Redacción del mensaje S/MIME definitivo requiriendo tanto firma como cifrado.

Luego, al recibir su respuesta que también viene firmada y cifrada, si abrimos el código fuente (figura 1.18), comprobamos que ya no se lee absolutamente nada en texto plano; todo el contenido del mensaje está encapsulado e ilegible dentro del bloque `application/pkcs7-mime`.

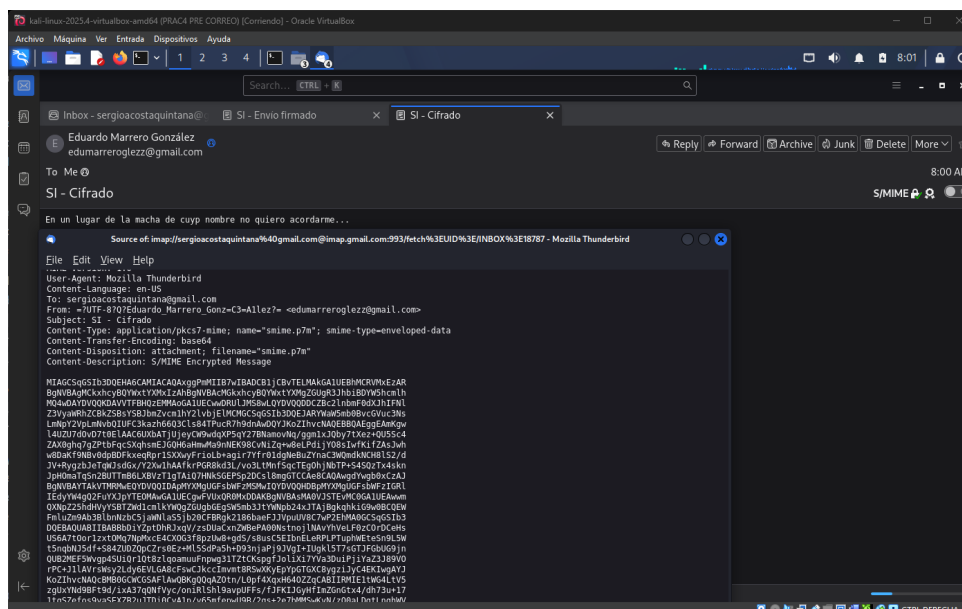


Figura 1.18: Código fuente del correo cifrado y firmado recibido como respuesta.

1.3. Decodificación con la utilidad “openssl smime” del mensaje cifrado y firmado que recibimos

En esta última parte vamos a analizar manualmente la estructura del correo seguro que nos mandó el compañero para entender cómo funciona por debajo y qué ha hecho Thunderbird. Para ello, primero exportamos el correo desde la aplicación en formato `.eml` para poder trabajarlo con OpenSSL desde la consola de Kali. Como vemos en la figura 1.19, hacemos un `ls` rápido para comprobar que tenemos en el directorio tanto el correo (`correo_cifrado.eml`) como el certificado del compañero (`cert_comp.crt`).

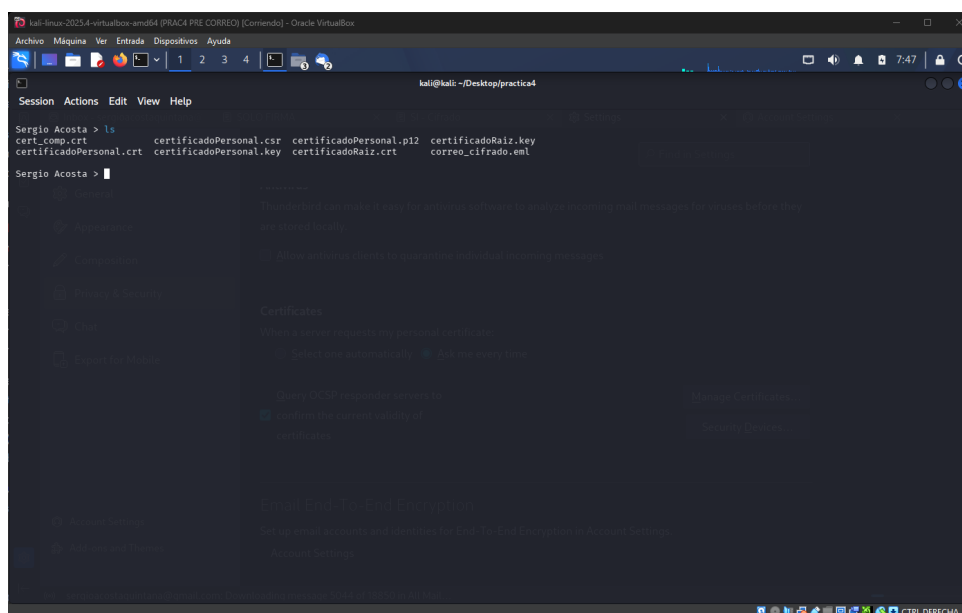


Figura 1.19: Verificación de los ficheros exportados en el directorio de trabajo.

El primer paso es quitarle la capa exterior de cifrado al mensaje. Para descifrar el mensaje ejecutamos el siguiente comando de OpenSSL (seccionado línea a línea para mayor legibilidad):

```
openssl smime -decrypt \  
-in correo_cifrado.eml \  
-recip certificadoPersonal.crt \  
-inkey certificadoPersonal.key \  
-out correo_descifrado.p7s
```

Al hacerlo nos sale un fichero nuevo (**correo_descifrado.p7s**) que ya no está cifrado, pero que sigue teniendo la firma digital incrustada (es de tipo **multipart/signed**). En la figura 1.20 se puede ver un volcado (**head**) de las primeras líneas del resultado para comprobar cómo queda.

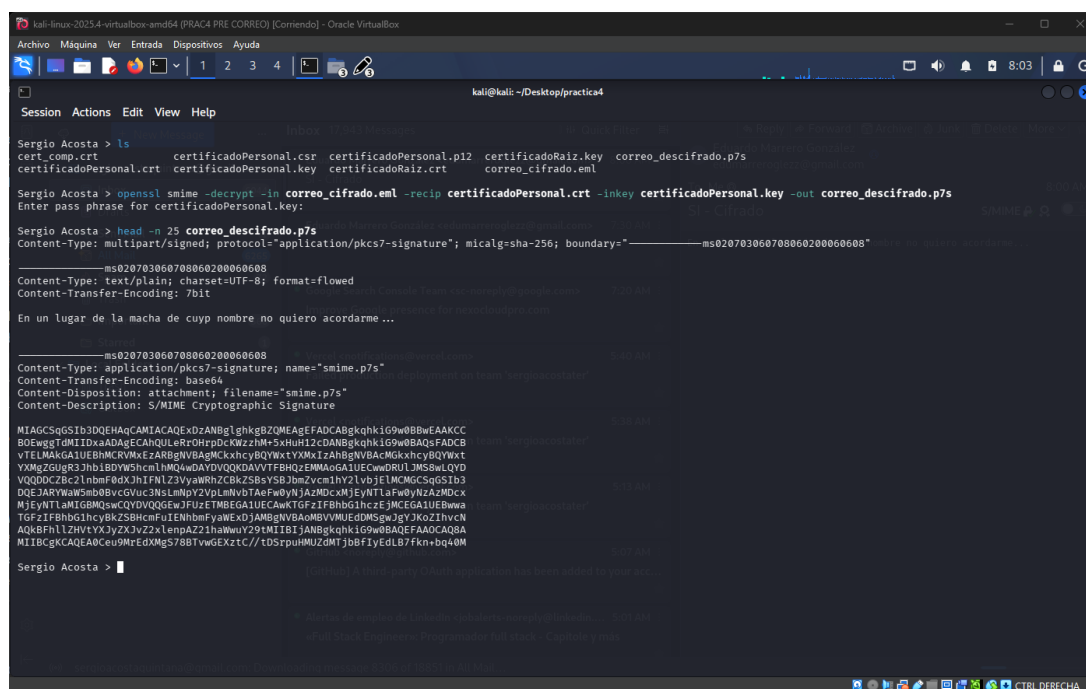


Figura 1.20: Descifrado del sobre digital y muestra de las cabeceras del archivo temporal firmado.

En este proceso se utiliza criptografía híbrida. Básicamente, el mensaje original se cifró con una clave simétrica generada aleatoriamente, y esta clave se cifró a su vez con la clave pública del destinatario (en este caso, nosotros). De esta forma, solo el destinatario que posee su clave privada puede descifrar el mensaje y acceder a esa clave simétrica que desbloquea el contenido real.

Teniendo ya ese documento intermedio descifrado (`correo_descifrado.p7s`), nos toca comprobar precisamente que la firma incrustada es válida. Para ello, ejecutamos el siguiente comando para verificarla:

```
openssl smime -verify \  
-in correo_descifrado.p7s \  
-certfile cert_comp.crt \  
-CAfile certificadoRaiz.crt \  
-out texto_final.txt
```

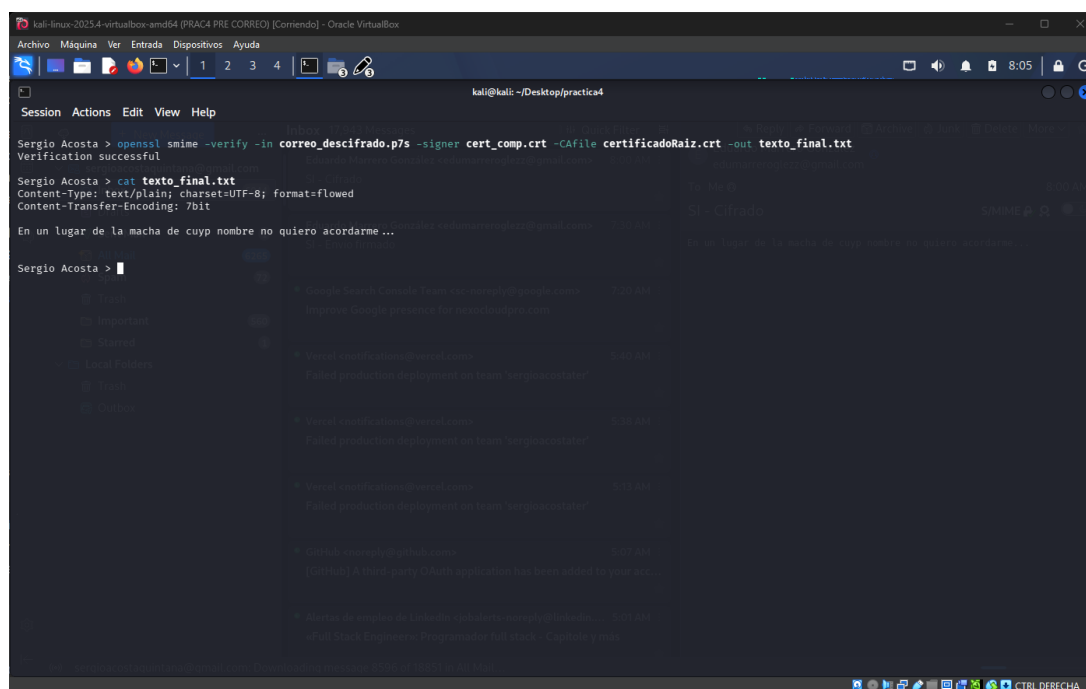


Figura 1.21: Verificación exitosa de la firma digital y extracción definitiva del cuerpo del mensaje.

Como vemos en la figura 1.21, al ejecutar el comando nos da el mensaje “Verification successful”, que confirma que la firma es válida, que el correo es verdaderamente suyo y que nadie lo ha modificado por el camino. En este punto, OpenSSL ha usado la clave pública del remitente y el certificado de la autoridad para hacer las matemáticas de validación. Además, la consola nos extrae el texto limpio y lo guarda en el archivo `texto_final.txt`.

Aquí dejo el volcado de lo que traía el correo original por dentro en texto plano:

```
Content-Type: text/plain; charset=UTF-8; format=flowed
Content-Transfer-Encoding: 7bit
```

En un lugar de la macha de cuyp nombre no quiero acordarme...

Conclusión

Con esta práctica se ha podido comprobar el funcionamiento del estándar S/MIME para el envío de correos electrónicos seguros. Se ha generado un certificado personal firmado por una autoridad de certificación, se ha configurado el cliente de correo Thunderbird para firmar y cifrar mensajes, y finalmente se ha analizado y decodificado manualmente un mensaje S/MIME mediante la herramienta OpenSSL. Esto nos ha permitido comprender tanto el uso práctico de estos mecanismos a nivel de usuario como su funcionamiento interno a nivel criptográfico.

Bibliografía

- [1] The OpenSSL Project, “OpenSSL Documentation – smime command,” [En línea]. Disponible: <https://www.openssl.org/docs/manmaster/man1/openssl-smime.html>.
- [2] CICEI, “OpenSSL Howto,” [En línea]. Disponible: <https://openssl.cicei.com>.
- [3] Mozilla Foundation, “Thunderbird Support – End-to-End Encryption and S/MIME,” [En línea]. Disponible: <https://support.mozilla.org>.
- [4] E. Rescorla, “S/MIME Version 3 Message Specification,” RFC 2633, Internet Engineering Task Force (IETF), 2001.
- [5] OpenAI, “ChatGPT,” Herramienta de IA utilizada como apoyo en la redacción y revisión del documento, [En línea]. Disponible: <https://openai.com/chat>.